

PAULT DOCUMENTATION

2026 02 19 Pro Features

Source: docs/plans/2026-02-19-pro-features.md

Theme: Clean Technical

Generated: February 26, 2026

Pault Pro Features Implementation Plan

> **For Claude:** REQUIRED SUB-SKILL: Use superpowers:executing-plans to implement this plan task-by-task.

Goal: Add a Pro tier to Pault covering four pillars: AI Assist (improve/suggest/score prompts), API Runner (run prompts against LLMs in-app), Prompt Chains (sequential prompt pipelines with Shortcuts integration), and Sync (iCloud + Git-backed libraries), all gated behind StoreKit 2.

Architecture: Pro gating is built first as a shared ProStatusManager singleton; all other features check it before revealing UI. AI features share a single AIService actor and KeychainService; Chains introduce new SwiftData models (Chain, ChainStep) and an App Intents integration; Sync adds an optional CloudKit container and a new GitSyncManager shell-based service.

Tech Stack: SwiftUI + SwiftData (macOS 15+), StoreKit 2, CloudKit, App Intents (macOS 13+), URLSession async streaming, SecKeychain API, Process for git shell commands, XCTest for unit/integration tests

Design doc: docs/plans/2026-02-19-pro-features-design.md

Phase 0: Pro Gating Infrastructure

Task 1: KeychainService

Files:

- Create: Pault/Services/KeychainService.swift
- Create: PaultTests/Services/KeychainServiceTests.swift

Step 1: Write the failing tests

```
// PaultTests/Services/KeychainServiceTests.swift
import XCTest
@testable import Pault

final class KeychainServiceTests: XCTestCase {
    let service = KeychainService(service: "com.pault.test")

    override func tearDown() {
        service.delete(key: "test-key")
    }

    func test_saveAndLoad_returnsStoredValue() throws {
        try service.save(key: "test-key", value: "my-api-key")
        let loaded = service.load(key: "test-key")
        XCTAssertEqual(loaded, "my-api-key")
    }

    func test_overwrite_updatesValue() throws {
        try service.save(key: "test-key", value: "old")
        try service.save(key: "test-key", value: "new")
        XCTAssertEqual(service.load(key: "test-key"), "new")
    }
}
```

```
func test_delete_removesValue() throws {
try service.save(key: "test-key", value: "value")
service.delete(key: "test-key")
XCTAssertNil(service.load(key: "test-key"))
}

func test_loadMissing_returnsNil() {
XCTAssertNil(service.load(key: "nonexistent"))
}
}
```

Step 2: Run tests to verify they fail

```
cd /Users/dev/Documents/Software/macOS/Pault
xcodebuild test -scheme Pault -destination 'platform=macOS' -only-testing
PaultTests/KeychainServiceTests 2>&1 | grep -E "error:|FAILED|PASSED"
```

Expected: Build error — KeychainService not found.

Step 3: Implement KeychainService

```
// Pault/Services/KeychainService.swift
import Foundation
import Security

struct KeychainService {
let service: String

init(service: String = "com.pault.app") {
self.service = service
}

func save(key: String, value: String) throws {
let data = Data(value.utf8)
delete(key: key) // remove existing before add
let query: [CFString: Any] = [
kSecClass: kSecClassGenericPassword,
kSecAttrService: service,
kSecAttrAccount: key,
kSecValueData: data
]
let status = SecItemAdd(query as CFDictionary, nil)
guard status == errSecSuccess else {
throw KeychainError.saveFailed(status)
}
}

func load(key: String) -> String? {
let query: [CFString: Any] = [
kSecClass: kSecClassGenericPassword,
kSecAttrService: service,
kSecAttrAccount: key,
kSecReturnData: true,
kSecMatchLimit: kSecMatchLimitOne
]
var result: AnyObject?
let status = SecItemCopyMatching(query as CFDictionary, &result)
guard status == errSecSuccess, let data = result as? Data else { return nil }
return String(data: data, encoding: .utf8)
}
```

```

}

func delete(key: String) {
    let query: [CFString: Any] = [
        kSecClass: kSecClassGenericPassword,
        kSecAttrService: service,
        kSecAttrAccount: key
    ]
    SecItemDelete(query as CFDictionary)
}
}

enum KeychainError: Error {
    case saveFailed(OSStatus)
}

```

Step 4: Run tests to verify they pass

```

xcodebuild test -scheme Pault -destination 'platform=macOS' -only-testing
PaultTests/KeychainServiceTests 2>&1 | grep -E "error:|FAILED|PASSED"

```

Expected: 4 tests PASSED.

Step 5: Commit

```

git add Pault/Services/KeychainService.swift
PaultTests/Services/KeychainServiceTests.swift
git commit -m "feat: add KeychainService for secure API key storage"

```

Task 2: ProStatusManager

Files:

- Create: Pault/Services/ProStatusManager.swift
- Create: PaultTests/Services/ProStatusManagerTests.swift
- Create: Pault/StoreKit/Products.storekit (StoreKit config for local testing)

Step 1: Create StoreKit configuration file

In Xcode: File → New → File → StoreKit Configuration File → name it Products.storekit.

Define three products:

```

{
    "identifier": "com.pault.pro.monthly",
    "type": "autoRenewableSubscription",
    "displayName": "Pault Pro",
    "description": "AI Assist, API Runner, and Prompt Chains",
    "price": "9.99",
    "subscriptionGroupID": "pault_pro",
    "subscriptionPeriod": "P1M",
    "introductoryOffer": {
        "type": "freeTrial",
        "period": "P7D"
    }
}

```

Add `com.pault.pro.annual` (P1Y, 79.99) and `com.pault.team.monthly` (P1M, 19.99, group: `pault_team`) similarly.

Step 2: Write the failing tests

```
// PaultTests/Services/ProStatusManagerTests.swift
import XCTest
import StoreKit
@testable import Pault

@MainActor
final class ProStatusManagerTests: XCTestCase {
    func test_initialState_isNotPro() {
        let manager = ProStatusManager()
        XCTAssertFalse(manager.isProUnlocked)
        XCTAssertFalse(manager.isTeamUnlocked)
    }

    func test_proProductIDs_matchConfiguration() {
        XCTAssertEqual(ProStatusManager.proProductIDs, [
            "com.pault.pro.monthly",
            "com.pault.pro.annual"
        ])
        XCTAssertEqual(ProStatusManager.teamProductIDs, [
            "com.pault.team.monthly"
        ])
    }
}
```

Step 3: Implement ProStatusManager

```
// Pault/Services/ProStatusManager.swift
import Foundation
import StoreKit
import Observation

@MainActor
@Observable
final class ProStatusManager {
    static let shared = ProStatusManager()

    static let proProductIDs = [
        "com.pault.pro.monthly",
        "com.pault.pro.annual"
    ]
    static let teamProductIDs = [
        "com.pault.team.monthly"
    ]

    private(set) var isProUnlocked: Bool = false
    private(set) var isTeamUnlocked: Bool = false
    private(set) var availableProducts: [Product] = []

    private var transactionListener: Task<Void, Error>?

    init() {
        transactionListener = Task {
            await listenForTransactions()
        }
    }
}
```

```
Task { await refreshStatus() }
}

deinit {
transactionListener?.cancel()
}

func purchase(_ product: Product) async throws -> Bool {
let result = try await product.purchase()
switch result {
case .success(let verification):
let transaction = try verification.payloadValue
await transaction.finish()
await refreshStatus()
return true
case .userCancelled, .pending:
return false
@unknown default:
return false
}
}

func restorePurchases() async {
try? await AppStore.sync()
await refreshStatus()
}

func loadProducts() async {
availableProducts = (try? await Product.products(
for: Self.proProductIDs + Self.teamProductIDs
)) ?? []
}

private func refreshStatus() async {
var hasPro = false
var hasTeam = false
for await result in Transaction.currentEntitlements {
if let transaction = try? result.payloadValue {
if Self.proProductIDs.contains(transaction.productID) { hasPro = true }
if Self.teamProductIDs.contains(transaction.productID) { hasTeam = true }
}
}
isProUnlocked = hasPro
isTeamUnlocked = hasTeam
}

private func listenForTransactions() async {
for await result in Transaction.updates {
if let transaction = try? result.payloadValue {
await transaction.finish()
await refreshStatus()
}
}
}
}
```

Step 4: Run tests

```
xcodebuild test -scheme Pault -destination 'platform=macOS' -only-testing
PaultTests/ProStatusManagerTests 2>&1 | grep -E "error:|FAILED|PASSED"
```

Expected: 2 tests PASSED.

Step 5: Commit

```
git add Pault/Services/ProStatusManager.swift Pault/StoreKit/Products.storekit
PaultTests/Services/ProStatusManagerTests.swift
git commit -m "feat: add ProStatusManager with StoreKit 2 subscription support"
```

Task 3: PaywallView + ProBadge

Files:

- Create: Pault/Views/PaywallView.swift
- Create: Pault/Views/ProBadge.swift

Step 1: Implement ProBadge

```
// Pault/Views/ProBadge.swift
import SwiftUI

struct ProBadge: View {
    var body: some View {
        Text("PRO")
            .font(.system(size: 9, weight: .bold))
            .foregroundColor(.white)
            .padding(.horizontal, 5)
            .padding(.vertical, 2)
            .background(
                LinearGradient(
                    colors: [.purple, .blue],
                    startPoint: .leading,
                    endPoint: .trailing
                )
            )
            .clipShape(Capsule())
    }
}
```

Step 2: Implement PaywallView

```
// Pault/Views/PaywallView.swift
import SwiftUI
import StoreKit

struct PaywallView: View {
    let featureName: String
    let featureDescription: String
    let featureIcon: String
    @Environment(\.dismiss) private var dismiss
    @State private var proStatus = ProStatusManager.shared
    @State private var isLoading = false
    @State private var selectedProductID = "com.pault.pro.monthly"

    var body: some View {
        VStack(spacing: 24) {
            // Header
```

```
VStack(spacing: 12) {
  Image(systemName: featureIcon)
    .font(.system(size: 48))
    .foregroundColor(
      LinearGradient(colors: [.purple, .blue], startPoint: .top, endPoint: .bottom)
    )

  Text("Unlock \$(featureName)")
    .font(.title2)
    .fontWeight(.bold)

  Text(featureDescription)
    .font(.subheadline)
    .foregroundColor(.secondary)
    .multilineTextAlignment(.center)
    .frame(maxWidth: 300)
}

// Product picker
if !proStatus.availableProducts.isEmpty {
  Picker("Plan", selection: $selectedProductID) {
    ForEach(proStatus.availableProducts.filter {
      ProStatusManager.proProductIDs.contains($0.id)
    }) { product in
      Text("\$(product.displayName) – \$(product.displayPrice)").tag(product.id)
    }
  }
  .pickerStyle(.segmented)
  .frame(maxWidth: 320)
}

// CTA
Button {
  Task { await purchaseSelected() }
} label: {
  if isLoading {
    ProgressView().controlSize(.small)
  } else {
    Text("Start 7-Day Free Trial")
      .fontWeight(.semibold)
      .frame(maxWidth: .infinity)
  }
}
.buttonStyle(.borderedProminent)
.controlSize(.large)
.frame(maxWidth: 280)
.disabled(isLoading)

Button("Restore Purchases") {
  Task {
    await proStatus.restorePurchases()
    if proStatus.isProUnlocked { dismiss() }
  }
}
.font(.footnote)
.foregroundColor(.secondary)
.buttonStyle(.plain)
}
```



```
.padding(32)
.frame(width: 400, height: 420)
.task { await proStatus.loadProducts() }
}

private func purchaseSelected() async {
guard let product = proStatus.availableProducts.first(where: { $0.id ==
selectedProductID })
else { return }
isLoading = true
defer { isLoading = false }
let success = (try? await proStatus.purchase(product)) ?? false
if success { dismiss() }
}
}
```

Step 3: Verify previews build

Open PaywallView.swift in Xcode. Confirm the preview renders without build errors.

Step 4: Commit

```
git add Pault/Views/PaywallView.swift Pault/Views/ProBadge.swift
git commit -m "feat: add PaywallView and ProBadge for StoreKit gating"
```

Phase 1: AI Services

Task 4: AIService

Files:

- Create: Pault/Services/AIService.swift
- Create: PaultTests/Services/AIServiceTests.swift

Step 1: Define the AIService protocol and types

```
// Pault/Services/AIService.swift
import Foundation
import os

private let aiLogger = Logger(subsystem: "com.pault.app", category: "ai")

// MARK: - Types

struct AIConfig {
enum Provider: String, CaseIterable {
case claude, openai, ollama
var displayName: String {
switch self { case .claude: "Claude"; case .openai: "OpenAI"; case .ollama: "Ollama
(Local)" }
}
}
var provider: Provider
var model: String
var baseURL: String? // for ollama only

static let defaults: [Provider: String] = [
```

```

.claude: "claude-opus-4-6",
.openai: "gpt-4o",
.ollama: "llama3"
]
}

struct QualityScore {
let clarity: Int // 1-10
let specificity: Int
let roleDefinition: Int
let outputFormat: Int
let clarityReason: String
let specificityReason: String
let roleDefinitionReason: String
let outputFormatReason: String

var overall: Double {
Double(clarity + specificity + roleDefinition + outputFormat) / 4.0
}
}

struct VariableSuggestion {
let originalText: String // text to replace
let suggestedName: String // suggested {{variable_name}}
let reason: String
}

// MARK: - AIService Actor

actor AIService {
static let shared = AIService()

private let keychain = KeychainService()
private let session: URLSession

init(session: URLSession = .shared) {
self.session = session
}

// MARK: - Improve

func improve(content: String, instruction: String, config: AIConfig) async throws ->
String {
let systemPrompt = ""
You are an expert prompt engineer. The user will provide a prompt and an instruction.
Rewrite the prompt following the instruction exactly. Return ONLY the improved prompt
text, no commentary.
""
let userMessage = "Instruction: \n(instruction)\n\nPrompt: \n\n(content)"
return try await complete(system: systemPrompt, user: userMessage, config: config)
}

// MARK: - Suggest Variables

func suggestVariables(for content: String, config: AIConfig) async throws ->
[VariableSuggestion] {
let systemPrompt = ""
Analyze the following prompt and identify literal values that should be template

```

```

variables.
Return a JSON array of objects with keys: "originalText", "suggestedName", "reason".
Only suggest values that vary between uses. Return ONLY valid JSON.
"""
let response = try await complete(system: systemPrompt, user: content, config: config)
let data = Data(response.utf8)
let raw = try JSONDecoder().decode([[String: String]].self, from: data)
return raw.compactMap { dict in
guard let orig = dict["originalText"],
let name = dict["suggestedName"],
let reason = dict["reason"] else { return nil }
return VariableSuggestion(originalText: orig, suggestedName: name, reason: reason)
}
}

// MARK: - Auto-tag

func autoTag(content: String, existingTags: [String], config: AIConfig) async throws -> [String] {
let systemPrompt = """
Suggest 1-3 tags for the following prompt from this list:
\u{005C}(existingTags.joined(separator: ", ")).
If no existing tag fits, suggest a new short lowercase tag.
Return ONLY a JSON array of strings. Example: ["research","writing"]
"""
let response = try await complete(system: systemPrompt, user: content, config: config)
return (try? JSONDecoder().decode([String].self, from: Data(response.utf8))) ?? []
}

// MARK: - Quality Score

func qualityScore(for content: String, config: AIConfig) async throws -> QualityScore {
let systemPrompt = """
Rate the following prompt on four axes (1-10 each) and give one sentence of reasoning
per axis.
Return ONLY valid JSON with keys: clarity, specificity, roleDefinition, outputFormat,
clarityReason, specificityReason, roleDefinitionReason, outputFormatReason.
"""
let response = try await complete(system: systemPrompt, user: content, config: config)
let data = Data(response.utf8)
let dict = try JSONDecoder().decode([String: String].self, from: data)
return QualityScore(
clarity: Int(dict["clarity"] ?? "5") ?? 5,
specificity: Int(dict["specificity"] ?? "5") ?? 5,
roleDefinition: Int(dict["roleDefinition"] ?? "5") ?? 5,
outputFormat: Int(dict["outputFormat"] ?? "5") ?? 5,
clarityReason: dict["clarityReason"] ?? "",
specificityReason: dict["specificityReason"] ?? "",
roleDefinitionReason: dict["roleDefinitionReason"] ?? "",
outputFormatReason: dict["outputFormatReason"] ?? ""
)
}

// MARK: - Streaming Run

func streamRun(content: String, config: AIConfig) -> AsyncThrowingStream<String, Error>
{
AsyncThrowingStream { continuation in
Task {

```

```

do {
    let request = try buildRequest(
        system: "You are a helpful assistant.",
        user: content,
        config: config,
        stream: true
    )
    let (bytes, _) = try await session.bytes(for: request)
    for try await line in bytes.lines {
        guard line.hasPrefix("data: "),
            let data = line.dropFirst(6).data(using: .utf8),
            let token = parseStreamToken(data: data) else { continue }
        if token == "[DONE]" { break }
        continuation.yield(token)
    }
    continuation.finish()
} catch {
    continuation.finish(throwing: error)
}
}
}

// MARK: - Private Helpers

private func complete(system: String, user: String, config: AIConfig) async throws ->
String {
    let request = try buildRequest(system: system, user: user, config: config, stream:
false)
    let (data, response) = try await session.data(for: request)
    guard let http = response as? HTTPURLResponse, http.statusCode == 200 else {
        throw AIError.httpError((response as? HTTPURLResponse)?.statusCode ?? -1, data)
    }
    return try parseCompletionResponse(data: data, config: config)
}

private func buildRequest(system: String, user: String, config: AIConfig, stream: Bool)
throws -> URLRequest {
    let key = keychain.load(key: "ai.apikey.\(config.provider.rawValue)")
    guard let apiKey = key, !apiKey.isEmpty else { throw AIError.missingAPIKey }

    let url: URL
    let body: [String: Any]

    switch config.provider {
    case .claude:
        url = URL(string: "https://api.anthropic.com/v1/messages")!
        body = [
            "model": config.model,
            "max_tokens": 2048,
            "system": system,
            "messages": [["role": "user", "content": user]],
            "stream": stream
        ]
    case .openai, .ollama:
        let base = config.provider == .ollama ? (config.baseURL ?? "http://localhost:11434") :
"https://api.openai.com"
        url = URL(string: "\(base)/v1/chat/completions")!
    }
}

```

```

body = [
  "model": config.model,
  "messages": [
    ["role": "system", "content": system],
    ["role": "user", "content": user]
  ],
  "stream": stream
]
}

var request = URLRequest(url: url)
request.httpMethod = "POST"
request.setValue("application/json", forHTTPHeaderField: "Content-Type")
switch config.provider {
case .claude:
  request.setValue(apiKey, forHTTPHeaderField: "x-api-key")
  request.setValue("2023-06-01", forHTTPHeaderField: "anthropic-version")
case .openai, .ollama:
  request.setValue("Bearer \(apiKey)", forHTTPHeaderField: "Authorization")
}
request.httpBody = try JSONSerialization.data(withJSONObject: body)
return request
}

private func parseCompletionResponse(data: Data, config: AIConfig) throws -> String {
  let json = try JSONSerialization.jsonObject(with: data) as? [String: Any]
  switch config.provider {
  case .claude:
    if let content = (json?["content"] as? [[String: Any]])?.first,
       let text = content["text"] as? String { return text }
  case .openai, .ollama:
    if let choices = json?["choices"] as? [[String: Any]],
       let message = choices.first?["message"] as? [String: Any],
       let text = message["content"] as? String { return text }
  }
  throw AIError.parseError(data)
}

private func parseStreamToken(data: Data) -> String? {
  guard let json = try? JSONSerialization.jsonObject(with: data) as? [String: Any] else {
    return nil
  }
  // Claude streaming
  if let delta = json["delta"] as? [String: Any], let text = delta["text"] as? String {
    return text
  }
  // OpenAI streaming
  if let choices = json["choices"] as? [[String: Any]],
     let delta = choices.first?["delta"] as? [String: Any],
     let text = delta["content"] as? String { return text }
  return nil
}
}

enum AIError: LocalizedError {
  case missingAPIKey
  case httpError(Int, Data)
  case parseError(Data)

  var errorDescription: String? {

```

```

switch self {
case .missingAPIKey: return "No API key configured. Add one in Settings → AI."
case .httpError(let code, _): return "API request failed with status \(code)."
case .parseError: return "Could not parse the AI response."
}
}
}

```

Step 2: Write tests (using mock URLSession)

```

// PaulTests/Services/AIServiceTests.swift
import XCTest
@testable import Pault

final class AIServiceTests: XCTestCase {

    func test_missingAPIKey_throwsMissingAPIKey() async throws {
        // KeychainService returns nil for unknown keys
        let service = AIService()
        let config = AIConfig(provider: .openai, model: "gpt-4o")
        do {
            _ = try await service.improve(content: "test", instruction: "improve", config: config)
            XCTFail("Expected throw")
        } catch AIError.missingAPIKey {
            // expected
        }
    }

    func test_qualityScore_parsesAllFields() throws {
        let json = """
{"clarity": "8", "specificity": "7", "roleDefinition": "6", "outputFormat": "9",
"clarityReason": "Clear.", "specificityReason": "Specific.", "roleDefinitionReason": "Has
role.", "outputFormatReason": "Formatted."}
""".data(using: .utf8)!
        // This tests the parsing logic independently – wire up if AIService exposes
        parseQualityJSON
        // For now verify types exist
        XCTAssertNotNil(json)
    }
}

```

Step 3: Run tests

```

xcodebuild test -scheme Pault -destination 'platform=macOS' -only-testing
PaulTests/AIServiceTests 2>&1 | grep -E "error:|FAILED|PASSED"

```

Expected: 2 tests PASSED.

Step 4: Commit

```

git add Pault/Services/AIService.swift PaulTests/Services/AIServiceTests.swift
git commit -m "feat: add AIService with improve, suggest-variables, auto-tag,
quality-score, and streaming"

```

Task 5: Settings → AI Tab

Files:

- Modify: Pault/PreferencesView.swift — add AI tab

Step 1: Add AISettingsTab to PreferencesView

In Pault/PreferencesView.swift, add after the existing DataTab() tab item:

```
AISettingsTab()
.tabItem {
Label("AI", systemImage: "sparkles")
}
```

And add the AISettingsTab private struct at the bottom of the file (before #Preview):

```
// MARK: - AI Settings Tab

private struct AISettingsTab: View {
    @AppStorage("aiProvider") private var aiProvider: String = "claude"
    @AppStorage("aiModel") private var aiModel: String = "claude-opus-4-6"
    @AppStorage("aiOllamaURL") private var ollamaURL: String = "http://localhost:11434"

    @State private var apiKey: String = ""
    @State private var showKey: Bool = false
    @State private var keychain = KeychainService()
    @State private var savedIndicator: Bool = false

    private var selectedProvider: AIConfig.Provider {
        AIConfig.Provider(rawValue: aiProvider) ?? .claude
    }

    var body: some View {
        Form {
            Section("Provider") {
                Picker("Provider", selection: $aiProvider) {
                    ForEach(AIConfig.Provider.allCases, id: \.rawValue) { p in
                        Text(p.displayName).tag(p.rawValue)
                    }
                }
                .onChange(of: aiProvider) { _, newProvider in
                    aiModel = AIConfig.defaults[AIConfig.Provider(rawValue: newProvider) ?? .claude] ?? ""
                    loadKey()
                }

                if selectedProvider == .ollama {
                    TextField("Base URL", text: $ollamaURL)
                        .font(.system(.body, design: .monospaced))
                }
            }

            Section("API Key") {
                HStack {
                    if showKey {
                        TextField("Paste API key...", text: $apiKey)
                    } else {
                        SecureField("Paste API key...", text: $apiKey)
                    }
                }
                Button(showKey ? "Hide" : "Show") { showKey.toggle() }
                    .buttonStyle(.plain)
                    .foregroundColor(.tint)
            }
        }
    }
}
```

```

HStack {
  Button("Save Key") {
    try? keychain.save(key: "ai.apikey.\(aiProvider)", value: apiKey)
    savedIndicator = true
    DispatchQueue.main.asyncAfter(deadline: .now() + 2) { savedIndicator = false }
  }
  if savedIndicator {
    Label("Saved", systemImage: "checkmark.circle.fill")
      .font(.caption)
      .foregroundColor(.green)
  }
}

Section("Model") {
  TextField("Model name", text: $aiModel)
    .font(.system(.body, design: .monospaced))
  Text("e.g. claude-opus-4-6, gpt-4o, llama3")
    .font(.caption)
    .foregroundColor(.secondary)
}
.padding()
.onAppear { loadKey() }
}

private func loadKey() {
  apiKey = keychain.load(key: "ai.apikey.\(aiProvider)") ?? ""
}

```

Step 2: Increase preferences frame height

Update the frame in `PreferencesView.body`:

```
.frame(width: 450, height: 380) // was 320
```

Step 3: Build and verify

Open Preferences (Cmd+,) → confirm AI tab appears → enter a dummy key → Save Key → green checkmark appears → close and reopen → key persists.

Step 4: Commit

```

git add Pault/PreferencesView.swift
git commit -m "feat: add AI settings tab with provider, API key (Keychain), and model selection"

```

Phase 2: AI Assist Panel

Task 6: AIAssistPanel View

Files:

- Create: `Pault/Views/AIAssistPanel.swift`
- Create: `Pault/Views/QualityScoreView.swift`

Step 1: Implement QualityScoreView

```
// Pault/Views/QualityScoreView.swift
import SwiftUI

struct QualityScoreView: View {
    let score: QualityScore

    var body: some View {
        VStack(alignment: .leading, spacing: 12) {
            HStack {
                Text("Quality Score")
                .font(.headline)
                Spacer()
                Text(String(format: "%.1f / 10", score.overall))
                .font(.title3)
                .fontWeight(.bold)
                .foregroundColor(scoreColor(score.overall))
            }

            ForEach(axes, id: \.label) { axis in
                VStack(alignment: .leading, spacing: 2) {
                    HStack {
                        Text(axis.label)
                        .font(.subheadline)
                        .fontWeight(.medium)
                        Spacer()
                        Text("\(axis.value)/10")
                        .font(.caption)
                        .foregroundColor(.secondary)
                    }
                    ProgressView(value: Double(axis.value), total: 10)
                    .tint(scoreColor(Double(axis.value)))
                    Text(axis.reason)
                    .font(.caption)
                    .foregroundColor(.secondary)
                }
            }
        }
        .padding()
    }

    private var axes: [(label: String, value: Int, reason: String)] {
        [
            ("Clarity", score.clarity, score.clarityReason),
            ("Specificity", score.specificity, score.specificityReason),
            ("Role Definition", score.roleDefinition, score.roleDefinitionReason),
            ("Output Format", score.outputFormat, score.outputFormatReason)
        ]
    }

    private func scoreColor(_ value: Double) -> Color {
        value >= 8 ? .green : value >= 5 ? .orange : .red
    }
}
```

Step 2: Implement AIAssistPanel

```
// Pault/Views/AIAssistPanel.swift
import SwiftUI
import AppKit
```

```
struct AIAssistPanel: View {
@Binding var content: String
@AppStorage("aiProvider") private var aiProvider: String = "claude"
@AppStorage("aiModel") private var aiModel: String = "claude-opus-4-6"
@State private var selectedTab = 0
@State private var instruction = ""
@State private var improvedContent: String? = nil
@State private var variableSuggestions: [VariableSuggestion] = []
@State private var tagSuggestions: [String] = []
@State private var qualityScore: QualityScore? = nil
@State private var isLoading = false
@State private var errorMessage: String? = nil

private var config: AIConfig {
AIConfig(
provider: AIConfig.Provider(rawValue: aiProvider) ?? .claude,
model: aiModel
)
}

var body: some View {
VStack(spacing: 0) {
Picker("", selection: $selectedTab) {
Label("Improve", systemImage: "wand.and.stars").tag(0)
Label("Variables", systemImage: "curlybraces").tag(1)
Label("Tags", systemImage: "tag").tag(2)
Label("Score", systemImage: "chart.bar").tag(3)
}
.pickerStyle(.segmented)
.padding(8)

Divider()

Group {
switch selectedTab {
case 0: improveTab
case 1: variablesTab
case 2: tagsTab
case 3: scoreTab
default: EmptyView()
}
}
.frame(maxWidth: .infinity, maxHeight: .infinity)

if let error = errorMessage {
HStack {
Image(systemName: "exclamationmark.triangle.fill").foregroundColor(.orange)
Text(error).font(.caption).foregroundColor(.secondary)
Spacer()
Button("Dismiss") { errorMessage = nil }.font(.caption).buttonStyle(.plain)
}
.padding(8)
.background(Color.orange.opacity(0.1))
}
}
.frame(height: 200)
.background(.background)
.clipShape(RoundedRectangle(cornerRadius: 8))
}
```

```

.overlay(RoundedRectangle(cornerRadius: 8).stroke(Color.separator, lineWidth: 0.5))
}

// MARK: - Improve Tab
private var improveTab: some View {
VStack(spacing: 8) {
if let improved = improvedContent {
ScrollView {
Text(improved)
.font(.body)
.frame(maxWidth: .infinity, alignment: .leading)
.padding(8)
}
HStack {
Button("Accept") { content = improved; improvedContent = nil }
.buttonStyle(.borderedProminent)
Button("Reject") { improvedContent = nil }
.buttonStyle(.bordered)
Spacer()
Button("Try Again") { Task { await runImprove() } }
.buttonStyle(.plain)
}
.padding(.horizontal, 8)
.padding(.bottom, 8)
} else {
HStack {
TextField("Instruction (e.g. Make more specific)", text: $instruction)
.textFieldStyle(.roundedBorder)
Button {
Task { await runImprove() }
} label: {
if isLoading { ProgressView().controlSize(.small) }
else { Image(systemName: "arrow.right.circle.fill") }
}
.disabled(instruction.isEmpty || isLoading)
}
.padding(8)
Spacer()
}
}
}

// MARK: - Variables Tab
private var variablesTab: some View {
VStack {
if variableSuggestions.isEmpty {
Button(isLoading ? "Analyzing..." : "Suggest Variables") {
Task { await runSuggestVariables() }
}
.buttonStyle(.borderedProminent)
.disabled(isLoading)
Spacer()
} else {
List(variableSuggestions, id: \.originalText) { s in
HStack {
VStack(alignment: .leading) {
Text("\(s.originalText)").font(.caption).foregroundColor(.secondary)
Text("→ {{\s.suggestedName}}").font(.caption).fontWeight(.medium)

```

```
}
Spacer()
Button("Apply") {
content = content.replacingOccurrences(of: s.originalText, with:
"{{$\s.suggestedName}}")
}.buttonStyle(.bordered)
}
}
.listStyle(.plain)
}
}
.padding(8)
}

// MARK: - Tags Tab
private var tagsTab: some View {
VStack {
if tagSuggestions.isEmpty {
Button(isLoading ? "Analyzing..." : "Suggest Tags") {
Task { await runAutoTag() }
}
.buttonStyle(.borderedProminent)
.disabled(isLoading)
Spacer()
} else {
HStack {
ForEach(tagSuggestions, id: \.self) { tag in
Text(tag)
.font(.caption)
.padding(.horizontal, 10)
.padding(.vertical, 4)
.background(Color.accentColor.opacity(0.15))
.clipShape(Capsule())
}
Spacer()
}
.padding(8)
Spacer()
}
}
}

// MARK: - Score Tab
private var scoreTab: some View {
Group {
if let score = qualityScore {
ScrollView { QualityScoreView(score: score) }
} else {
VStack {
Button(isLoading ? "Scoring..." : "Score This Prompt") {
Task { await runQualityScore() }
}
.buttonStyle(.borderedProminent)
.disabled(isLoading)
Spacer()
}
.padding(8)
}
}
```

```

}
}

// MARK: - Actions
private func runImprove() async {
    isLoading = true; errorMessage = nil
    do { improvedContent = try await AIService.shared.improve(content: content, instruction:
instruction, config: config) }
    catch { errorMessage = error.localizedDescription }
    isLoading = false
}

private func runSuggestVariables() async {
    isLoading = true; errorMessage = nil
    do { variableSuggestions = try await AIService.shared.suggestVariables(for: content,
config: config) }
    catch { errorMessage = error.localizedDescription }
    isLoading = false
}

private func runAutoTag() async {
    isLoading = true; errorMessage = nil
    do { tagSuggestions = try await AIService.shared.autoTag(content: content, existingTags:
[], config: config) }
    catch { errorMessage = error.localizedDescription }
    isLoading = false
}

private func runQualityScore() async {
    isLoading = true; errorMessage = nil
    do { qualityScore = try await AIService.shared.qualityScore(for: content, config:
config) }
    catch { errorMessage = error.localizedDescription }
    isLoading = false
}
}
}

```

Step 3: Wire into EditPromptView

In `Pault/Views/EditPromptView.swift`:

1. Add `@State private var showAIAssist = false` property
2. Add `@State private var proStatus = ProStatusManager.shared`
3. Add `@State private var showPaywall = false`
4. In the toolbar, add after existing toolbar items:

```

ToolbarItem(placement: .primaryAction) {
    Button {
        if proStatus.isProUnlocked {
            showAIAssist.toggle()
        } else {
            showPaywall = true
        }
    } label: {
        Label("AI Assist", systemImage: "sparkles")
    }
}

```

```
.overlay(alignment: .topTrailing) {
if !proStatus.isProUnlocked { ProBadge().offset(x: 6, y: -6) }
}
}
```

5. Below the RichTextEditor in the VStack, add:

```
if showAIAssist {
AIAssistPanel(content: $promptContent)
.padding(.horizontal)
.transition(.move(edge: .bottom).combined(with: .opacity))
}
```

6. Add sheet modifier:

```
.sheet(isPresented: $showPaywall) {
PaywallView(
featureName: "AI Assist",
featureDescription: "Improve prompts, suggest variables, and score quality using AI.",
featureIcon: "sparkles"
)
}
```

Step 4: Build and verify

Build. Open a prompt in edit mode → sparkles toolbar button present. If not Pro: paywall sheet. If Pro: AI Assist panel slides in below editor.

Step 5: Commit

```
git add Pault/Views/AIAssistPanel.swift Pault/Views/QualityScoreView.swift
Pault/Views/EditPromptView.swift
git commit -m "feat: add AI Assist panel to EditPromptView with Improve, Variables,
Tags, and Score"
```

Phase 3: API Runner

Task 7: ResponsePanel + Run Button

Files:

- Create: Pault/Views/ResponsePanel.swift
- Modify: Pault/Views/PromptDetailView.swift

Step 1: Implement ResponsePanel

```
// Pault/Views/ResponsePanel.swift
import SwiftUI

struct ResponsePanel: View {
@Binding var response: String
@Binding var isStreaming: Bool
var onSaveAsNewPrompt: (() -> Void)?
var onCancel: (() -> Void)?

var body: some View {
VStack(alignment: .leading, spacing: 8) {
HStack {
```

```

Label(isStreaming ? "Generating..." : "Response", systemImage: "text.bubble")
    .font(.caption)
    .foregroundStyle(.secondary)
Spacer()
if isStreaming {
    Button("Cancel") { onCancel?() }
        .font(.caption)
        .buttonStyle(.plain)
        .foregroundStyle(.red)
    } else {
        HStack(spacing: 8) {
            Button("Copy") {
                NSPasteboard.general.clearContents()
                NSPasteboard.general.setString(response, forType: .string)
            }
                .font(.caption)
                .buttonStyle(.bordered)

            Button("Save as New Prompt") { onSaveAsNewPrompt?() }
                .font(.caption)
                .buttonStyle(.bordered)
        }
    }
}

ScrollView {
    Text(response.isEmpty ? "..." : response)
        .font(.body)
        .frame(maxWidth: .infinity, alignment: .leading)
        .textSelection(.enabled)
    }
    .frame(maxHeight: 200)
    .padding(8)
    .background(Color.secondary.opacity(0.05))
    .clipShape(RoundedRectangle(cornerRadius: 6))
    }
    .padding()
    .background(.background)
    .clipShape(RoundedRectangle(cornerRadius: 8))
    .overlay(RoundedRectangle(cornerRadius: 8).stroke(Color.separator, lineWidth: 0.5))
    }
}

```

Step 2: Modify PromptDetailView to add Run button + ResponsePanel

In Pault/Views/PromptDetailView.swift, add:

```

// New state properties
@State private var showResponsePanel = false
@State private var streamingResponse = ""
@State private var isRunning = false
@State private var runTask: Task<Void, Never>? = nil
@State private var runError: String? = nil
@State private var showPaywall = false
@State private var proStatus = ProStatusManager.shared
@AppStorage("aiProvider") private var aiProvider: String = "claude"
@AppStorage("aiModel") private var aiModel: String = "claude-opus-4-6"

```

In the action buttons area (alongside the Copy button), add a Run button:

```

Button {
  if proStatus.isProUnlocked {
    Task { await runPrompt() }
  } else {
    showPaywall = true
  }
  label: {
    Label("Run", systemImage: "play.fill")
  }
  .buttonStyle(.borderedProminent)
  .overlay(alignment: .topTrailing) {
    if !proStatus.isProUnlocked { ProBadge().offset(x: 6, y: -6) }
  }
}

```

Below the existing detail content, add:

```

if showResponsePanel {
  ResponsePanel(
    response: $streamingResponse,
    isStreaming: $isRunning,
    onSaveAsNewPrompt: { saveResponseAsNewPrompt() },
    onCancel: { runTask?.cancel(); isRunning = false }
  )
  .padding(.horizontal)
}

```

And add the run method and sheet:

```

private func runPrompt() async {
  let resolved = TemplateEngine.resolve(prompt.content, variables:
    prompt.templateVariables)
  let config = AIConfig(
    provider: AIConfig.Provider(rawValue: aiProvider) ?? .claude,
    model: aiModel
  )
  showResponsePanel = true
  streamingResponse = ""
  isRunning = true
  runTask = Task {
    do {
      for try await token in await AIService.shared.streamRun(content: resolved, config:
        config) {
        streamingResponse += token
      }
    } catch {
      runError = error.localizedDescription
    }
    isRunning = false
  }
  await runTask?.value
}

private func saveResponseAsNewPrompt() {
  // Create new prompt with streaming response as content
  // Reuse PromptService.createPrompt or insert directly via modelContext
}

.sheet(isPresented: $showPaywall) {
  PaywallView(
    featureName: "API Runner",

```



```
featureDescription: "Run your prompts directly against any LLM and see the response
inline.",
featureIcon: "play.fill"
)
}
```

Step 3: Build and verify

Build. Open a prompt detail → Run button visible (with PRO badge if not subscribed). Click Run (with API key configured) → ResponsePanel appears → text streams in → Copy button works.

Step 4: Commit

```
git add Pault/Views/ResponsePanel.swift Pault/Views/PromptDetailView.swift
git commit -m "feat: add API Runner with streaming response panel in PromptDetailView"
```

Phase 4: Prompt Chains

Task 8: Chain Data Models

Files:

- Create: Pault/Models/Chain.swift
- Modify: Pault/PaultApp.swift — register Chain/ChainStep in ModelContainer

Step 1: Implement Chain + ChainStep models

```
// Pault/Models/Chain.swift
import Foundation
import SwiftData

enum ChainBinding: Codable, Hashable {
case literal(String)
case previousOutput
case userInput
}

@Model
final class Chain {
var id: UUID
var name: String
var createdAt: Date
var updatedAt: Date
@Relationship(deleteRule: .cascade) var steps: [ChainStep]

init(name: String) {
self.id = UUID()
self.name = name
self.steps = []
self.createdAt = Date()
self.updatedAt = Date()
}
}

@Model
final class ChainStep {
var id: UUID
```

```

var sortOrder: Int
var variableBindingsJSON: String // JSON-encoded [String: ChainBinding]
var prompt: Prompt?
var chain: Chain?

init(prompt: Prompt, sortOrder: Int, bindings: [String: ChainBinding] = [:]) {
    self.id = UUID()
    self.prompt = prompt
    self.sortOrder = sortOrder
    self.chain = nil
    self.variableBindingsJSON = (try? String(data: JSONEncoder().encode(bindings), encoding:
        .utf8)) ?? "{}"
}

var variableBindings: [String: ChainBinding] {
    get {
        (try? JSONDecoder().decode([String: ChainBinding].self, from:
            Data(variableBindingsJSON.utf8))) ?? [:]
    }
    set {
        variableBindingsJSON = (try? String(data: JSONEncoder().encode(newValue), encoding:
            .utf8)) ?? "{}"
    }
}
}

```

Step 2: Register in ModelContainer

In `Pault/PaultApp.swift`, update the `sharedModelContainer`:

```

let schema = Schema([Prompt.self, Tag.self, TemplateVariable.self, Chain.self,
    ChainStep.self])

```

Step 3: Write model tests

```

// PaultTests/Models/ChainTests.swift
import XCTest
import SwiftData
@testable import Pault

final class ChainTests: XCTestCase {
    func test_chainStep_bindingRoundTrip() {
        let prompt = Prompt(title: "Test", content: "Hello {{name}}")
        let step = ChainStep(prompt: prompt, sortOrder: 0, bindings: [
            "name": .literal("Alice"),
            "context": .previousOutput
        ])
        let bindings = step.variableBindings
        XCTAssertEqual(bindings["name"], .literal("Alice"))
        XCTAssertEqual(bindings["context"], .previousOutput)
    }
}

```

Step 4: Run tests

```

xcodebuild test -scheme Pault -destination 'platform=macOS' -only-testing
    PaultTests/ChainTests 2>&1 | grep -E "error:|FAILED|PASSED"

```

Expected: 1 test PASSED.

Step 5: Commit

```
git add Pault/Models/Chain.swift Pault/PaultApp.swift PaultTests/Models/ChainTests.swift
git commit -m "feat: add Chain and ChainStep SwiftData models"
```

Task 9: ChainRunner Service**Files:**

- Create: Pault/Services/ChainRunner.swift
- Create: PaultTests/Services/ChainRunnerTests.swift

Step 1: Implement ChainRunner

```
// Pault/Services/ChainRunner.swift
import Foundation

actor ChainRunner {
    static let shared = ChainRunner()

    /// Executes a chain sequentially, returning the final step's output.
    func run(chain: Chain, userInputs: [String: String] = [:]) async throws -> String {
        let sortedSteps = chain.steps.sorted { $0.sortOrder < $1.sortOrder }
        guard !sortedSteps.isEmpty else { throw ChainError.emptyChain }

        var previousOutput = ""
        let config = buildConfig()

        for step in sortedSteps {
            guard let prompt = step.prompt else { throw ChainError.missingPrompt(step.id) }
            let resolved = resolveStep(step: step, prompt: prompt, previousOutput: previousOutput,
                                      userInputs: userInputs)
            var stepOutput = ""
            for try await token in await AIService.shared.streamRun(content: resolved, config:
                                                                    config) {
                stepOutput += token
            }
            previousOutput = stepOutput
        }

        return previousOutput
    }

    private func resolveStep(step: ChainStep, prompt: Prompt, previousOutput: String,
                             userInputs: [String: String]) -> String {
        var content = prompt.content
        let bindings = step.variableBindings
        for variable in prompt.templateVariables {
            let binding = bindings[variable.name] ?? .literal(variable.defaultValue)
            let value: String
            switch binding {
            case .literal(let s): value = s
            case .previousOutput: value = previousOutput
            case .userInput: value = userInputs[variable.name] ?? variable.defaultValue
            }
            content = content.replacingOccurrences(of: "{$\\(variable.name)}", with: value)
        }
        return content
    }
}
```

```

}

private func buildConfig() -> AIConfig {
    let provider = UserDefaults.standard.string(forKey: "aiProvider") ?? "claude"
    let model = UserDefaults.standard.string(forKey: "aiModel") ?? "claude-opus-4-6"
    return AIConfig(provider: AIConfig.Provider(rawValue: provider) ?? .claude, model:
model)
}
}

enum ChainError: LocalizedError {
    case emptyChain
    case missingPrompt(UUID)

    var errorDescription: String? {
        switch self {
            case .emptyChain: return "This chain has no steps."
            case .missingPrompt(let id): return "Step \(id) references a deleted prompt."
        }
    }
}

```

Step 2: Commit

```

git add Pault/Services/ChainRunner.swift
git commit -m "feat: add ChainRunner service for sequential prompt execution"

```

Task 10: Chain Editor UI

Files:

- Create: Pault/Views/ChainEditorView.swift
- Modify: Pault/SidebarView.swift — add Chains section

Step 1: Implement ChainEditorView

```

// Pault/Views/ChainEditorView.swift
import SwiftUI
import SwiftData

struct ChainEditorView: View {
    @Bindable var chain: Chain
    @Environment(\.modelContext) private var modelContext
    @Query(sort: [SortDescriptor(\Prompt.updatedAt, order: .reverse)]) private var
allPrompts: [Prompt]
    @State private var showAddStep = false
    @State private var runOutput: String? = nil
    @State private var isRunning = false
    @State private var runError: String? = nil
    @State private var showPaywall = false
    @State private var proStatus = ProStatusManager.shared

    var body: some View {
        VStack(alignment: .leading, spacing: 0) {
            // Header
            HStack {
                Text(chain.name)
                .font(.title2)

```

```
.fontWeight(.semibold)
Spacer()
Button {
    if proStatus.isProUnlocked {
        Task { await runChain() }
    } else {
        showPaywall = true
    }
    label: {
        if isRunning {
            ProgressView().controlSize(.small)
        } else {
            Label("Run Chain", systemImage: "play.fill")
        }
    }
    .buttonStyle(.borderedProminent)
    .disabled(chain.steps.isEmpty || isRunning)
    .overlay(alignment: .topTrailing) {
        if !proStatus.isProUnlocked { ProBadge().offset(x: 6, y: -6) }
    }
    .padding()

Divider()

// Steps list
if chain.steps.isEmpty {
    ContentUnavailableView {
        Label("No Steps", systemImage: "list.bullet")
    } description: {
        Text("Add prompts to build your chain.")
    } actions: {
        Button("Add First Step") { showAddStep = true }
        .buttonStyle(.borderedProminent)
    }
} else {
    List {
        ForEach(chain.steps.sorted { $0.sortOrder < $1.sortOrder }) { step in
            ChainStepRow(step: step) {
                modelContext.delete(step)
            }
        }
        .onMove { indices, newOffset in
            var sorted = chain.steps.sorted { $0.sortOrder < $1.sortOrder }
            sorted.move(fromOffsets: indices, toOffset: newOffset)
            for (i, step) in sorted.enumerated() { step.sortOrder = i }
        }
        .listStyle(.plain)
    }

// Add step button
Button("+ Add Step") { showAddStep = true }
    .buttonStyle(.plain)
    .foregroundColor(.tint)
    .padding()

// Output
```

```

if let output = runOutput {
    Divider()
    ResponsePanel(
        response: .constant(output),
        isStreaming: .constant(false),
        onSaveAsNewPrompt: nil
    )
    .padding()
}
}

.sheet(isPresented: $showAddStep) {
    PromptPickerSheet(onSelect: { prompt in
        let step = ChainStep(prompt: prompt, sortOrder: chain.steps.count)
        step.chain = chain
        chain.steps.append(step)
        modelContext.insert(step)
    })
}

.sheet(isPresented: $showPaywall) {
    PaywallView(
        featureName: "Prompt Chains",
        featureDescription: "Chain prompts together into powerful pipelines.",
        featureIcon: "arrow.triangle.branch"
    )
}

private func runChain() async {
    isRunning = true
    runOutput = nil
    do {
        runOutput = try await ChainRunner.shared.run(chain: chain)
    } catch {
        runError = error.localizedDescription
    }
    isRunning = false
}

private struct ChainStepRow: View {
    @Bindable var step: ChainStep
    var onDelete: () -> Void

    var body: some View {
        HStack {
            Image(systemName: "line.3.horizontal")
                .foregroundColor(.secondary)
            VStack(alignment: .leading) {
                Text(step.prompt?.title ?? "Deleted prompt")
                    .font(.headline)
                Text("\((step.variableBindings.count) variable bindings)")
                    .font(.caption)
                    .foregroundColor(.secondary)
            }
            Spacer()
            Button("Remove", role: .destructive) { onDelete() }
                .buttonStyle(.plain)
                .foregroundColor(.red)
        }
    }
}

```

```

        .font(.caption)
    }
    .padding(.vertical, 4)
    }
    }

    private struct PromptPickerSheet: View {
        @Environment(\.dismiss) private var dismiss
        @Query(sort: [SortDescriptor(\Prompt.updatedAt, order: .reverse)]) private var prompts:
        [Prompt]
        var onSelect: (Prompt) -> Void

        var body: some View {
            VStack {
                Text("Add Step").font(.headline).padding()
                List(prompts) { prompt in
                    Button(prompt.title.isEmpty ? "Untitled" : prompt.title) {
                        onSelect(prompt)
                        dismiss()
                    }
                    .buttonStyle(.plain)
                }
                Button("Cancel") { dismiss() }.padding()
            }
            .frame(width: 300, height: 400)
        }
    }

```

Step 2: Add Chains section to SidebarView

In `Pault/SidebarView.swift`, after the existing filter rows and before the Divider:

```

// Chains section header + list
if !chains.isEmpty {
    Divider().padding(.vertical, 4)
    HStack {
        Text("Chains").font(.caption).fontWeight(.semibold).foregroundColor(.secondary)
        Spacer()
        Button { /* create new chain */ } label: { Image(systemName: "plus") }
        .buttonStyle(.plain)
    }
    .padding(.horizontal, 12)
    ForEach(chains) { chain in
        FilterRow(title: chain.name, icon: "arrow.triangle.branch", isSelected: false) {
            // navigate to chain editor
        }
    }
    .padding(.horizontal, 8)
}

```

Add `@Query private var chains: [Chain]` to `SidebarView`.

Step 3: Commit

```

git add Pault/Views/ChainEditorView.swift Pault/SidebarView.swift
git commit -m "feat: add Chain Editor UI with step management and run support"

```

Task 11: Shortcuts App Intents

Files:

- Create: `Pault/Intents/RunChainIntent.swift`
- Create: `Pault/Intents/CopyPromptIntent.swift`

Step 1: Implement `RunChainIntent`

```
// Pault/Intents/RunChainIntent.swift
import AppIntents
import SwiftData

@available(macOS 13, *)
struct RunChainIntent: AppIntent {
    static var title: LocalizedStringResource = "Run Prompt Chain"
    static var description = IntentDescription("Run a Pault prompt chain and return the final output.")

    @Parameter(title: "Chain Name")
    var chainName: String

    @Parameter(title: "Initial Input", default: "")
    var initialInput: String

    func perform() async throws -> some ReturnsValue<String> {
        // Fetch chain from SwiftData
        let container = try ModelContainer(for: Chain.self, ChainStep.self, Prompt.self, Tag.self, TemplateVariable.self)
        let context = ModelContext(container)
        let descriptor = FetchDescriptor<Chain>(predicate: #Predicate { $0.name == chainName })
        guard let chain = try context.fetch(descriptor).first else {
            throw IntentError.chainNotFound(chainName)
        }
        let output = try await ChainRunner.shared.run(
            chain: chain,
            userInputs: initialInput.isEmpty ? [] : ["input": initialInput]
        )
        return .result(value: output)
    }
}

enum IntentError: LocalizedError {
    case chainNotFound(String)
    var errorDescription: String? {
        if case .chainNotFound(let name) = self { return "No chain named '\(name)' found." }
        return nil
    }
}
```

Step 2: Implement `CopyPromptIntent`

```
// Pault/Intents/CopyPromptIntent.swift
import AppIntents
import SwiftData
import AppKit

@available(macOS 13, *)
struct CopyPromptIntent: AppIntent {
    static var title: LocalizedStringResource = "Copy Prompt"
```



```

static var description = IntentDescription("Copy a Pault prompt to the clipboard
(resolves template variables).")

@Parameter(title: "Prompt Title")
var promptTitle: String

func perform() async throws -> some ReturnValue<String> {
    let container = try ModelContainer(for: Prompt.self, Tag.self, TemplateVariable.self)
    let context = ModelContext(container)
    let descriptor = FetchDescriptor<Prompt>(predicate: #Predicate { $0.title == promptTitle
    })
    guard let prompt = try context.fetch(descriptor).first else {
        throw IntentError.chainNotFound(promptTitle)
    }
    let resolved = TemplateEngine.resolve(prompt.content, variables:
    prompt.templateVariables)
    await MainActor.run {
        NSPasteboard.general.clearContents()
        NSPasteboard.general.setString(resolved, forType: .string)
    }
    return .result(value: resolved)
}
}

```

Step 3: Commit

```

git add Pault/Intents/RunChainIntent.swift Pault/Intents/CopyPromptIntent.swift
git commit -m "feat: add RunChainIntent and CopyPromptIntent for Shortcuts.app
integration"

```

Phase 5: Sync

Task 12: iCloud Sync Toggle

Files:

- Modify: Pault/PaultApp.swift — conditionally enable CloudKit
- Modify: Pault/PreferencesView.swift — Sync tab

Step 1: Update ModelContainer for optional CloudKit

In Pault/PaultApp.swift, replace the sharedModelContainer computed property:

```

static var sharedModelContainer: ModelContainer = {
    let schema = Schema([Prompt.self, Tag.self, TemplateVariable.self, Chain.self,
    ChainStep.self])
    let useCloud = UserDefaults.standard.bool(forKey: "iCloudSyncEnabled")
    let config: ModelConfiguration
    if useCloud {
        config = ModelConfiguration(schema: schema, cloudKitDatabase:
        .private("iCloud.com.pault.app"))
    } else {
        config = ModelConfiguration(schema: schema, isStoredInMemoryOnly: false)
    }
    return try! ModelContainer(for: schema, configurations: [config])
}()

```

> Note: Because `sharedModelContainer` is a static property initialized once at launch, changing the iCloud toggle requires an app restart. Show an alert informing the user.

Step 2: Add Sync tab to PreferencesView

```
SyncSettingsTab()
  .tabItem {
    Label("Sync", systemImage: "arrow.triangle.2.circlepath")
  }
```

Add `SyncSettingsTab`:

```
// MARK: - Sync Settings Tab

private struct SyncSettingsTab: View {
  @AppStorage("iCloudSyncEnabled") private var iCloudSyncEnabled: Bool = false
  @State private var showRestartAlert = false

  var body: some View {
    Form {
      Section("iCloud") {
        Toggle("Enable iCloud Sync", isOn: Binding(
          get: { iCloudSyncEnabled },
          set: { newValue in
            iCloudSyncEnabled = newValue
            showRestartAlert = true
          }
        ))
        Text("Syncs your prompts across all Macs signed into the same iCloud account. Restart
        Paul to apply changes.")
          .font(.caption)
          .foregroundColor(.secondary)
      }

      Section("Git Library") {
        Text("Configure a Git-backed library to sync with GitHub, GitLab, or any remote.")
          .font(.caption)
          .foregroundColor(.secondary)
        GitSyncSection()
      }
    }
    .padding()
    .alert("Restart Required", isPresented: $showRestartAlert) {
      Button("Restart Now") { NSApp.terminate(nil) }
      Button("Later", role: .cancel) { }
    } message: {
      Text("Pault needs to restart to apply sync settings.")
    }
  }
}
```

Step 3: Commit

```
git add Pault/PaultApp.swift Pault/PreferencesView.swift
git commit -m "feat: add iCloud sync toggle with CloudKit-backed ModelContainer"
```

Task 13: GitSyncManager + PromptMarkdownSerializer

Files:

- Create: `Pault/Services/PromptMarkdownSerializer.swift`
- Create: `Pault/Services/GitSyncManager.swift`
- Create: `PaultTests/Services/PromptMarkdownSerializerTests.swift`

Step 1: Implement PromptMarkdownSerializer

```
// Pault/Services/PromptMarkdownSerializer.swift
import Foundation

struct PromptMarkdownSerializer {

    static func encode(prompt: Prompt) -> String {
        var lines = ["---"]
        lines.append("id: \"\$(prompt.id.uuidString)\"")
        lines.append("title: \"\$(prompt.title.isEmpty ? \"\" : \"\$(prompt.title)\")")
        let tagNames = prompt.tags.map { $0.name }
        lines.append("tags: \"\$(tagNames.map { \"\$(\$0)\" }.joined(separator: ", "))")
        lines.append("favorite: \"\$(prompt.isFavorite)\"")
        if !prompt.templateVariables.isEmpty {
            lines.append("variables:")
            for v in prompt.templateVariables.sorted(by: { $0.sortOrder < $1.sortOrder }) {
                lines.append("  - name: \"\$(v.name)\"")
                lines.append("    default: \"\$(v.defaultValue)\"")
            }
        }
        lines.append("---")
        lines.append("")
        lines.append(prompt.content)
        return lines.joined(separator: "\n")
    }

    /// Returns (id, title, tags, favorite, variables, content)
    static func decode(_ text: String) -> (id: UUID?, title: String, tagNames: [String],
        favorite: Bool, variables: [(name: String, default: String)], content: String)? {
        let parts = text.components(separatedBy: "\n---\n")
        guard parts.count >= 2 else { return nil }
        let frontmatter = parts[0].replacingOccurrences(of: "---\n", with: "")
        let content = parts.dropFirst().joined(separator: "\n---\n").trimmingCharacters(in:
            .whitespacesAndNewlines)

        var id: UUID? = nil
        var title = ""
        var tagNames: [String] = []
        var favorite = false
        var variables: [(String, String)] = []

        for line in frontmatter.components(separatedBy: "\n") {
            if line.hasPrefix("id:") {
                let raw = line.replacingOccurrences(of: "id:", with: "").trimmingCharacters(in:
                    .whitespaces).trimmingCharacters(in: .init(charactersIn: "\""))
                id = UUID(uuidString: raw)
            } else if line.hasPrefix("title:") {
                title = line.replacingOccurrences(of: "title:", with: "").trimmingCharacters(in:
                    .whitespaces).trimmingCharacters(in: .init(charactersIn: "\""))
            } else if line.hasPrefix("favorite:") {
                favorite = line.contains("true")
            } else if line.hasPrefix("tags:") {

```

```

let raw = line.replacingOccurrences(of: "tags:", with: "").trimmingCharacters(in:
.whitespaces)
tagNames = raw.trimmingCharacters(in: .init(charactersIn: "["))
.components(separatedBy: ",")
.map { $0.trimmingCharacters(in: .whitespaces).trimmingCharacters(in:
.init(charactersIn: "\"")) }
.filter { !$0.isEmpty }
}
}

return (id, title, tagNames, favorite, variables, content)
}
}

```

Step 2: Write serializer tests

```

// PaultTests/Services/PromptMarkdownSerializerTests.swift
import XCTest
@testable import Pault

final class PromptMarkdownSerializerTests: XCTestCase {

func test_decode_parsesIdTitleContent() throws {
let md = ""
---
id: "550E8400-E29B-41D4-A716-446655440000"
title: "My Prompt"
tags: ["writing", "research"]
favorite: true
---

Summarize this {{text}}
""

let result = PromptMarkdownSerializer.decode(md)
XCTAssertNotNil(result)
XCTAssertEqual(result?.title, "My Prompt")
XCTAssertEqual(result?.tagNames, ["writing", "research"])
XCTAssertTrue(result?.favorite ?? false)
XCTAssertEqual(result?.content, "Summarize this {{text}}")
XCTAssertEqual(result?.id?.uuidString.lowercased(),
"550e8400-e29b-41d4-a716-446655440000")
}
}

```

Step 3: Run tests

```

xcodebuild test -scheme Pault -destination 'platform=macOS' -only-testing
PaultTests/PromptMarkdownSerializerTests 2>&1 | grep -E "error:|FAILED|PASSED"

```

Expected: 1 test PASSED.

Step 4: Implement GitSyncManager

```

// Pault/Services/GitSyncManager.swift
import Foundation
import SwiftData
import os

private let gitLogger = Logger(subsystem: "com.pault.app", category: "gitsync")

```

```

actor GitSyncManager {
static let shared = GitSyncManager()

// MARK: - Pull (repo → SwiftData)
func pull(from repoURL: URL, into context: ModelContext) async throws -> (added: Int,
updated: Int) {
try ensureGitRepo(at: repoURL)
let promptsDir = repoURL.appendingPathComponent("prompts")
let files = try FileManager.default.contentsOfDirectory(
at: promptsDir, includingPropertiesForKeys: nil
).filter { $0.pathExtension == "md" }

var added = 0, updated = 0
for file in files {
let text = try String(contentsOf: file, encoding: .utf8)
guard let parsed = PromptMarkdownSerializer.decode(text) else { continue }
if let existingID = parsed.id {
let descriptor = FetchDescriptor<Prompt>(predicate: #Predicate { $0.id == existingID })
if let existing = try context.fetch(descriptor).first {
existing.title = parsed.title
existing.content = parsed.content
existing.isFavorite = parsed.favorite
existing.updatedAt = Date()
updated += 1
} else {
insertPrompt(parsed: parsed, into: context)
added += 1
}
} else {
insertPrompt(parsed: parsed, into: context)
added += 1
}
}
try context.save()
return (added, updated)
}

// MARK: - Push (SwiftData → repo)
func push(prompts: [Prompt], to repoURL: URL, remote: String?) async throws {
let promptsDir = repoURL.appendingPathComponent("prompts")
try FileManager.default.createDirectory(at: promptsDir, withIntermediateDirectories:
true)

for prompt in prompts {
let content = PromptMarkdownSerializer.encode(prompt: prompt)
let file = promptsDir.appendingPathComponent("\(prompt.id.uuidString).md")
try content.write(to: file, atomically: true, encoding: .utf8)
}

try await shell("git", args: ["-C", repoURL.path, "add", "-A"])
try await shell("git", args: ["-C", repoURL.path, "commit", "-m", "Pault sync
\((ISO8601DateFormatter().string(from: Date()))")])
if let remote {
try await shell("git", args: ["-C", repoURL.path, "push", remote, "HEAD"])
}
gitLogger.info("push: Pushed \(prompts.count) prompts to \(repoURL.lastPathComponent)")
}

```

```

// MARK: - Helpers
private func ensureGitRepo(at url: URL) throws {
    let gitDir = url.appendingPathComponent(".git")
    if !FileManager.default.fileExists(atPath: gitDir.path) {
        try FileManager.default.createDirectory(at: url, withIntermediateDirectories: true)
        _ = try? shellSync("git", args: ["-C", url.path, "init"])
    }
}

private func insertPrompt(parsed: (id: UUID?, title: String, tagNames: [String],
favorite: Bool, variables: [(name: String, default: String)], content: String), into
context: ModelContext) {
    let prompt = Prompt(title: parsed.title, content: parsed.content)
    if let id = parsed.id { prompt.id = id }
    prompt.isFavorite = parsed.favorite
    context.insert(prompt)
}

@discardableResult
private func shell(_ command: String, args: [String]) async throws -> String {
    try await withCheckedThrowingContinuation { continuation in
        let process = Process()
        process.executableURL = URL(fileURLWithPath: "/usr/bin/env")
        process.arguments = [command] + args
        let pipe = Pipe()
        process.standardOutput = pipe
        process.standardError = pipe
        do {
            try process.run()
            process.waitUntilExit()
            let output = String(data: pipe.fileHandleForReading.readDataToEndOfFile(), encoding:
                .utf8) ?? ""
            if process.terminationStatus == 0 {
                continuation.resume(returning: output)
            } else {
                continuation.resume(throwing: GitError.commandFailed(output))
            }
        } catch {
            continuation.resume(throwing: error)
        }
    }
}

private func shellSync(_ command: String, args: [String]) throws -> String {
    let process = Process()
    process.executableURL = URL(fileURLWithPath: "/usr/bin/env")
    process.arguments = [command] + args
    let pipe = Pipe()
    process.standardOutput = pipe
    try process.run()
    process.waitUntilExit()
    return String(data: pipe.fileHandleForReading.readDataToEndOfFile(), encoding: .utf8) ??
        ""
}

enum GitError: LocalizedError {
    case commandFailed(String)
}

```

```

var errorDescription: String? {
    if case .commandFailed(let output) = self { return "Git command failed: \(output)" }
    return nil
}
}

```

Step 5: Add GitSyncSection to Sync tab

In `PreferencesView.swift`, implement `GitSyncSection`:

```

private struct GitSyncSection: View {
    @AppStorage("gitRepoPath") private var repoPath: String = ""
    @AppStorage("gitRemoteURL") private var remoteURL: String = ""
    @Environment(\.modelContext) private var modelContext
    @Query private var prompts: [Prompt]
    @State private var syncStatus: String? = nil
    @State private var isSyncing = false

    var body: some View {
        VStack(alignment: .leading, spacing: 8) {
            HStack {
                TextField("Repository path", text: $repoPath)
                    .font(.system(.body, design: .monospaced))
                Button("Browse...") { selectRepoFolder() }
            }

            TextField("Remote URL (optional)", text: $remoteURL)
                .font(.system(.body, design: .monospaced))

            HStack(spacing: 8) {
                Button("Pull") {
                    Task { await doPull() }
                }
                .disabled(repoPath.isEmpty || isSyncing)

                Button("Push") {
                    Task { await doPush() }
                }
                .disabled(repoPath.isEmpty || isSyncing)

                if isSyncing { ProgressView().controlSize(.small) }
                if let status = syncStatus {
                    Text(status).font(.caption).foregroundColor(.secondary)
                }
            }
        }

        private func selectRepoFolder() {
            let panel = NSOpenPanel()
            panel.canChooseFiles = false
            panel.canChooseDirectories = true
            panel.allowsMultipleSelection = false
            if panel.runModal() == .OK, let url = panel.url {
                repoPath = url.path
            }
        }

        private func doPull() async {

```

```

guard let url = URL(string: "file://\(repoPath)") else { return }
isSyncing = true
do {
    let (added, updated) = try await GitSyncManager.shared.pull(from: url, into:
modelContext)
    syncStatus = "Pulled: +\(added) new, ~\(updated) updated"
} catch {
    syncStatus = "Error: \(error.localizedDescription)"
}
isSyncing = false
}

private func doPush() async {
guard let url = URL(string: "file://\(repoPath)") else { return }
isSyncing = true
do {
    let remote = remoteURL.isEmpty ? nil : remoteURL
    try await GitSyncManager.shared.push(prompts: prompts, to: url, remote: remote)
    syncStatus = "Pushed \(prompts.count) prompts"
} catch {
    syncStatus = "Error: \(error.localizedDescription)"
}
isSyncing = false
}
}

```

Step 6: Commit

```

git add Pault/Services/PromptMarkdownSerializer.swift
Pault/Services/GitSyncManager.swift Pault/PreferencesView.swift
PaultTests/Services/PromptMarkdownSerializerTests.swift
git commit -m "feat: add GitSyncManager and PromptMarkdownSerializer for Git-backed
library sync"

```

Phase 6: Pro Gating Wiring + Final Polish

Task 14: Wire Pro status to environment + final integration test

Files:

- Modify: Pault/PaultApp.swift — inject ProStatusManager into environment

Step 1: Inject ProStatusManager

In PaultApp.body, pass ProStatusManager.shared into the environment:

```

WindowGroup {
    ContentView()
        .tint(accentColor)
        .environment(ProStatusManager.shared)
        .onReceive(NotificationCenter.default.publisher(for: .openAboutWindow)) { _ in
            openWindow(id: "about")
        }
}

```

Step 2: Run full test suite


```
xcodebuild test -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E "Test Suite|FAILED|PASSED|error:"
```

Expected: All existing tests pass. New tests for KeychainService, ProStatusManager, AIService, ChainTests, PromptMarkdownSerializer all pass.

Step 3: Final commit

```
git add Pault/PaultApp.swift
git commit -m "feat: inject ProStatusManager into SwiftUI environment for app-wide Pro gating"
```

Verification Checklist

- [] **Pro Gating:** Tap AI Assist sparkles → paywall sheet appears → complete StoreKit sandbox purchase → feature becomes accessible → Restore Purchases → entitlement restored
- [] **AI Assist:** Edit a prompt → sparkles toolbar → Improve tab → enter instruction → improved version shown → Accept replaces content → Reject preserves original
- [] **Variable Suggestions:** AI Assist → Variables tab → Suggest → suggestion chips appear → Apply → {{variable}} inserted into content
- [] **API Runner:** Prompt detail → Run button → fill variables → streaming response appears → Cancel mid-stream → request aborted → Save as New Prompt → new prompt created
- [] **Prompt Chains:** Create 2-step chain → step 2 binding = previousOutput → Run Chain → step 2 receives step 1's output → final output displayed
- [] **Shortcuts:** Open Shortcuts.app → create automation → add "Run Prompt Chain" action → configure chain name → run → output returned
- [] **iCloud Sync:** Enable in Settings → Sync → restart → create prompt on Mac A → appears on Mac B within ~30s
- [] **Git Sync:** Push → verify .md files in repo directory → edit .md externally → Pull → SwiftData updated
- [] **API key security:** API key entered in Settings → AI → verify NOT stored in UserDefaults (defaults read com.pault.app) → verify stored in Keychain only